

C 和 C++ 输入输出方式整理

本文将全面整理 C 和 C++ 中的各种输入输出方式，讨论它们的使用方法、注意事项以及在使用时需要注意的细节。我们将对每种方式的使用进行详细的讨论，包括如何处理换行符、空白字符、格式化控制、传入参数、异常处理等。

C 语言的输入输出方式

1. getchar() 和 putchar()

1.1 getchar() - 从标准输入读取一个字符

- 使用方法:

getchar() 从标准输入读取一个字符（包括换行符），返回读取到的字符，若遇到文件结束符（EOF）或错误，返回 EOF。

```
char c;  
c = getchar(); // 从标准输入读取一个字符
```

- 注意事项:

- getchar() 会把换行符作为输入字符读取。如果需要逐字符读取而忽略换行符，需要额外处理。
- 需要检查 EOF，以处理输入结束或错误的情况。

1.2 putchar() - 输出一个字符

- 使用方法:

putchar() 输出一个字符到标准输出（通常是控制台），输出后会自动跳到下一行。

```
putchar('A'); // 输出字符 A
```

- 注意事项:

- putchar() 不会自动添加换行符，如果需要换行，需要显式地输出换行符 \n。

2. fgets() 和 fputs()

2.1 fgets() - 从标准输入读取一行

- 使用方法:

fgets() 读取一行输入，并保留换行符（如果存在），最多读取指定长度的字符。

```
char buffer[100];  
fgets(buffer, 100, stdin); // 从标准输入读取一行
```

- 注意事项:

- fgets() 会包括换行符，如果不希望它被读取，可以手动去除换行符。
- 如果输入的字符数超过指定的大小，fgets() 会读取并截断，但不会丢弃剩余的字符，这些字符会留在输入缓冲区中。

2.2 fputs() - 输出字符串

- 使用方法:

fputs() 输出一个字符串到指定的文件流，不会自动添加换行符。

```
fputs("Hello, World!", stdout); // 输出字符串
```

- 注意事项:

- fputs() 不会自动添加换行符，若需要换行，需要显式地调用 fputs("\n", stdout)。

3. scanf() 和 printf() (格式化输入输出)

3.1 scanf() - 格式化输入

- 使用方法:

scanf() 依据格式字符串从标准输入读取数据。

```
int num;
scanf("%d", &num); // 读取整数
```

- 注意事项:

- scanf() 默认跳过空白字符（空格、换行等），若输入数据包含空白字符且格式化字符不匹配，可能导致输入失败。
- 当读取字符串时，scanf() 会读取直到遇到第一个空白字符（空格、换行或制表符）为止。
- 需要检查返回值以确保读取成功，scanf() 返回读取的项数。

3.2 printf() - 格式化输出

- 使用方法:

printf() 依据格式字符串将数据输出到标准输出。

```
int num = 42;
printf("The number is: %d\n", num); // 输出格式化字符串
```

- 注意事项:

- printf() 支持多种格式化控制，如 %d（整数）、%s（字符串）、%f（浮点数）等。
- 需要注意格式化字符串的类型匹配，若格式字符与提供的数据类型不匹配，可能导致未定义行为。
- printf() 不会自动添加换行符，需要手动加上 \n。

4. fread() 和 fwrite() - 二进制文件读写

4.1 fread() - 从文件读取数据

- 使用方法:

fread() 从指定的文件流读取二进制数据。

```
int buffer[10];
size_t n = fread(buffer, sizeof(int), 10, file); // 读取 10 个整数
```

- 注意事项:

- fread() 不处理数据格式，只按字节读取数据，因此适用于二进制文件。
- 必须小心处理文件读写错误，通常需要检查返回值和 feof()、ferror() 来处理异常情况。

4.2 fwrite() - 向文件写入数据

- 使用方法:

fwrite() 将数据写入文件流。

```
int buffer[10] = {1, 2, 3};
fwrite(buffer, sizeof(int), 3, file); // 向文件写入 3 个整数
```

- 注意事项:

- 与 fread() 类似，fwrite() 处理的是原始数据，不进行格式化。

C 的替代 gets() 和 puts()

5. gets() - 不安全的输入函数

gets() 已被 C11 标准废弃，因为它不检查输入长度，容易导致缓冲区溢出。推荐使用 fgets() 来替代。

6. puts() - 输出字符串

puts() 会在输出后自动添加换行符。可以使用 fputs() 或 printf() 来代替，后者提供更多格式化控制。

C++ 中的输入输出方式

1. cin 和 cout

1.1 cin - 从标准输入读取

- 使用方法:

cin 用于从标准输入读取数据，自动处理空白字符。

```
int num;
cin >> num; // 从标准输入读取整数
```

- 注意事项:

- cin 会跳过空白字符（空格、换行符、制表符等），直到遇到下一个非空白字符。
- 如果输入的数据类型与变量不匹配，cin 会进入错误状态，导致后续输入失败。可以使用 cin.clear() 清除错误状态，并使用 cin.ignore() 清理输入缓冲区。

1.2 cout - 输出到标准输出

- 使用方法:

cout 用于输出数据，支持格式化输出。

```
int num = 42;
cout << "The number is: " << num << endl; // 输出整数并换行
```

- 注意事项:

- cout 不会自动添加换行符，需要手动加上 endl 或 "\n"。
- 支持链式输出，可以一次输出多个数据项。

2. ifstream、ofstream 和 fstream

2.1 ifstream - 从文件读取数据

- 使用方法:

ifstream 用于从文件中读取数据。

```
ifstream file("example.txt");
string line;
getline(file, line); // 从文件中读取一行
```

- 注意事项:

- 需要检查文件是否成功打开，可以通过 file.is_open() 来检查。
- getline() 会读取整行文本，直到遇到换行符。

2.2 ofstream - 向文件写入数据

- **使用方法:**

ofstream 用于将数据写入文件。

```
ofstream file("output.txt");  
file << "Hello, World!" << endl; // 向文件写入字符串并换行
```

- **注意事项:**

- 如果文件不存在，ofstream 会自动创建文件。
- 文件打开失败时需要处理异常，可以使用 try-catch 语句捕获文件打开失败的异常。

2.3 fstream - 读写文件

- **使用方法:**

fstream 用于同时进行文件的读取和写入操作。

```
fstream file("example.txt", ios::in | ios::out);  
file << "Data to file" << endl; // 写入数据
```

- **注意事项:**

- 需要指定合适的文件打开模式（如 ios::in、ios::out）。
- 确保文件在读取和写入时都没有被其他程序占用。

3. stringstream - 内存中的流操作

3.1 使用 stringstream 进行字符串流操作

- **使用方法:**

stringstream 可以将数据

转化为字符串或从字符串中提取数据。

```
stringstream ss;  
ss << 42 << " Hello!";  
string result = ss.str(); // 转换为字符串
```

- **注意事项:**

- stringstream 是一个内存中的流，不涉及实际的文件或标准输入输出。
- 可以方便地实现字符串与其他类型数据的转换。

总结

C 语言

- **推荐使用:**

- 输入: fgets() 代替 gets(), scanf() 用于格式化输入。
- 输出: fputs() 和 printf() 代替 puts()。

- **安全问题:** 使用 fgets() 来防止缓冲区溢出。

C++ 语言

- **推荐使用:**

- 输入: cin 和 getline(), cin 更适合读取格式化输入。

- 输出：cout 支持链式输出，ofstream 用于文件输出。
- **流的多功能性：** C++ 流类（如 fstream 和 stringstream ）提供了更丰富的功能，适合内存流和文件流操作。

通过选择适当的输入输出方式，程序可以在性能、安全性和可读性之间找到良好的平衡。

scanf / printf 和 cin / cout 格式控制方法

在 C 和 C++ 中，scanf 和 printf （C 标准库）以及 cin 和 cout （C++ 流式输入输出）都允许通过格式控制字符来指定输入输出的格式。以下是这两对函数常用的格式控制符及其对应的类型匹配字符。

C 语言

1. scanf 格式控制

scanf 用于格式化输入。它根据格式化字符串（包含占位符）从标准输入流中读取数据。

常见的 scanf 格式控制符

格式控制符	类型	描述
%d	int	读取一个十进制整数
%i	int	读取一个整数，可以是十进制、十六进制（以0x开头）或八进制（以0开头）
%u	unsigned int	读取一个无符号十进制整数
%f	float	读取一个浮点数
%lf	double	读取一个双精度浮点数
%c	char	读取一个字符（包括空白字符）
%s	char[]	读取一个字符串，遇到空白字符停止读取，自动添加终止符 \0
%x	unsigned int	读取一个十六进制整数
%o	unsigned int	读取一个八进制整数
%p	void *	读取一个指针，输出为 void * 类型（通常是内存地址）
%lf	double	读取一个双精度浮点数（注意，%f 对应 float ，但在 scanf 中，%f 和 %lf 都读取 double ）
%zu	size_t	读取一个size_t类型
%n	int*	读取当前输入位置的字符数，存储在对应的整数变量中（不读取数据）

scanf 类型匹配说明

- %d：用于读取十进制整数，并匹配 int 类型。
- %u：用于读取无符号整数。
- %f：用于读取浮点数，并匹配 float 类型。
- %lf：用于读取双精度浮点数，并匹配 double 类型。对于 scanf ， %f 和 %lf 都用于读取 double 类型，而对于 printf ， %f 用于 float ， %lf 用于 double 。
- %c：读取一个字符（即 char ）。
- %s：读取一个字符串，字符串以空白字符（空格、制表符或换行符）为分隔符。
- %p：读取一个指针，并返回其值（地址）。

注意事项：

- 使用 scanf 时，要确保传入变量的类型与格式控制符匹配，否则会导致未定义行为。

- 在读取字符串时，scanf 会遇到空格、制表符或换行符自动停止，并且不会读取空白字符。

2. printf 格式控制

printf 用于格式化输出，支持多种格式控制符，能够以特定格式输出各种数据类型。

常见的 printf 格式控制符

格式控制符	类型	描述
%d	int	输出十进制整数
%i	int	输出十进制整数（等同于 %d）
%u	unsigned int	输出无符号十进制整数
%f	float	输出浮点数，默认显示 6 位小数（需要使用 %.nf 控制精度）
%lf	double	输出双精度浮点数，通常与 %f 等价
%c	char	输出字符
%s	char[]	输出字符串（字符数组）
%x	unsigned int	输出十六进制整数（小写字母）
%X	unsigned int	输出十六进制整数（大写字母）
%o	unsigned int	输出八进制整数
%p	void*	输出指针值（通常为内存地址）
%zu	size_t	输出size_t类型
%e	float / double	输出浮点数，使用科学计数法显示（小写字母 e）
%E	float / double	输出浮点数，使用科学计数法显示（大写字母 E）
%g	float / double	根据数值自动选择输出 %e 或 %f，并移除多余的零
%G	float / double	与 %g 类似，但使用大写字母 E 代替 e
%n	int*	输出到当前输出位置的字符数（不输出数据）
%%	无类型	输出一个百分号字符 %

printf 格式化控制符的详细说明

- %d / %i**：输出十进制整数（与输入时的 %d 匹配）。
- %u**：输出无符号整数。
- %f**：输出浮点数，默认情况下输出 6 位小数。可以使用 %.nf 指定小数点后显示的位数。
- %s**：输出字符串，直到遇到空白字符（空格、换行符或制表符）为止。
- %x 和 %X**：输出无符号十六进制整数，%x 使用小写字母，%X 使用大写字母。
- %p**：输出指针（即地址），输出的是地址值。
- %n**：此格式控制符不输出数据，而是将当前输出的字符数存入指定的 int 变量中。
- %%**：输出百分号字符 %。

注意事项：

- 格式控制符与相应类型的匹配必须正确，若匹配错误，会导致未定义行为。
- printf 支持宽度和精度控制，可以指定输出的宽度（如 %10d）和小数点后的位数（如 %.2f）。
- 宽度控制**：例如 %10d 表示输出一个占据 10 个字符宽度的整数，如果实际输出小于 10，则会在左侧填充空格。
- 精度控制**：例如 %.2f 会显示浮点数的两位小数。

C++ 语言

1. cin 格式控制

C++ 中的 `cin` 使用流操作符 `>>` 从标准输入中读取数据。`cin` 会自动根据数据类型进行转换，处理输入时不需要显式指定格式控制符，但可以使用流操控符来调整格式。

常用的 cin 输入方法

控制符	描述
<code>>></code>	基本的输入操作符，从标准输入读取数据
<code>getline()</code>	从标准输入读取一行数据，包括空格，直到遇到换行符为止

注意事项：

- `cin` 默认会跳过输入中的空白字符（空格、换行等），直到遇到下一个非空白字符。
- 对于读取字符串时，`cin` 只会读取直到第一个空格，若想读取包含空格的整行，使用 `getline()`。

2. cout 格式控制

`cout` 使用流操作符 `<<` 输出数据。C++ 中的 `cout` 可以通过 `std::setw`、`std::setprecision` 等流操控符进行格式控制。

常用的 cout 格式控制符

控制符	描述
<code><<</code>	输出操作符，输出数据到标准输出
<code>std::setw()</code>	设置输出的最小宽度
<code>std::setprecision()</code>	设置浮点数输出的精度

|
`std::fixed`	设置输出为固定小数点格式
`std::scientific`	设置输出为科学计数法格式
`std::endl`	输出换行并刷新输出流
`std::flush`	刷新输出流（不换行）

示例：

```
#include <iostream>
#include <iomanip> // for std::setw, std::setprecision

int main() {
    int a = 42;
    double b = 3.14159;

    // 设置最小宽度
    std::cout << std::setw(10) << a << std::endl;

    // 设置浮点数的精度
    std::cout << std::setprecision(3) << b << std::endl;

    // 使用固定格式输出浮点数
    std::cout << std::fixed << b << std::endl;

    return 0;
}
```

注意事项：

- 使用 `std::setw` 设置宽度时，输出内容不会自动填充空白，必须明确使用 `std::setfill` 来指定填充字符。
- `std::setprecision()` 控制输出浮点数的小数位数，但如果没有设置 `std::fixed`，它会影响输出数字的总位数。

总结

C 语言：

- `scanf` 和 `printf` 支持丰富的格式控制符，能够精准地控制输入和输出的格式。
- 格式符和数据类型的匹配非常重要，错误的匹配会导致未定义行为。

C++ 语言：

- `cin` 和 `cout` 使用流操作符与操控符来控制格式。
- 支持通过流操控符设置精度、宽度、对齐方式等，比 C 语言的 `printf / scanf` 更加灵活和易用。